BIRLA INSTITUTE OF TECHNOLOGY & SCIENCE, PILANI

Work Integrated Learning Programmes Division

Post Graduate Program

in

Artificial Intelligence and Machine Learning

Demand Forecasting for Retail Inventory Optimization

CAPSTONE PROJECT

Submitted in partial fulfilment of the requirements of the

Post Graduate Certification Program
in
Artificial Intelligence and Machine Learning

By

SAJAL DEBNATH (2024aiml521)
ANTONY SUDHARSAN ( )
KRITI SHARMA ( )
AJAY ( )

Under the supervision of

PROF. SUDARSHAN S. DESHMUKH

Project work carried out at

BIRLA INSTITUTE OF TECHNOLOGY AND SCIENCE
Pilani (Rajasthan) INDIA
(April 2026)

PGCP AIML_Oct_2024_C6 CAPSTONE PROJECT

Demand Forecasting for Retail Inventory Optimization

Submitted in partial fulfilment of the requirements of the

PGCP - Artificial Intelligence and Machine Learning

By

SAJAL DEBNATH (2024aiml521)
ANTONY SUDHARSAN ( )
KRITI SHARMA ( )
AJAY ( )

Under the supervision of

PROF. SUDARSHAN S. DESHMUKH

Project work carried out at

BIRLA INSTITUTE OF TECHNOLOGY AND SCIENCE
PILANI (RAJASTHAN)
(April 2026)

BIRLA INSTITUTE OF TECHNOLOGY AND SCIENCE, PILANI

CERTIFICATE

This is to certify that the Capstone Project entitled Digital Twin for Indian Rural Health Care Infrastructure submitted by

SAJAL DEBNATH (2024aiml521)
ANTONY SUDHARSAN ( )
KRITI SHARMA ( )
AJAY ( )

in partial fulfilment of the requirements of AIML Oct 2024_PCAMZC321 Capstone Project, embodies the work done by him/her under my supervision.

Place : _____ Signature of the Mentor

Date : _____ Name : _____

BIRLA INSTITUTE OF TECHNOLOGY & SCIENCE, PILANI

SECOND SEMESTER 2024-25

AIML Oct 2024_PCAMZC321 CAPSTONE PROJECT

Project Title : Demand Forecasting for Retail Inventory Optimization

Name of Mentor : **PROF. SUDARSHAN S. DESHMUKH**

Name of Student : SAJAL DEBNATH, ANTONY SUDHARSAN, KRITI SHARMA, AJAY

ID No. of Student : 2024AIML521, < >, < >, < >

Abstract

Retail inventory optimization is a multi-dimensional problem characterized by demand uncertainty, price elasticity, seasonal variability, and supply chain constraints. Traditional heuristic-based approaches often fail to capture the complex, non-linear relationships between these factors, leading to suboptimal stocking decisions. This project addresses the problem by designing a data-driven **Retail Intelligence System** that leverages machine learning for simultaneous stocking priority classification and sales forecasting.

The proposed system integrates heterogeneous data sources, including product attributes, pricing signals, promotional indicators, supply chain variables, environmental conditions, and digital engagement metrics. A comprehensive data preprocessing and feature engineering pipeline is developed to transform raw inputs into a structured representation suitable for modeling. Notably, a hybrid feature construction strategy is employed, combining numerical variables with contextual representations derived from categorical attributes through a templated encoding mechanism. This approach enables the model to capture both quantitative relationships and semantic context within the data.

Two primary predictive tasks are formulated:

- A **multi-class classification problem** for determining stocking priority levels (High, Medium, Low)
- A **regression problem** for estimating expected product demand (sales forecasting)

For the classification task, a **hybrid Logistic Regression model** is implemented, leveraging both engineered numerical features and contextual embeddings. For the regression task, supervised learning models are trained to capture demand patterns influenced by pricing, promotions, and external factors. Model performance is evaluated using standard metrics, including accuracy for classification and R^2 and RMSE for regression, demonstrating strong predictive capability and generalization.

In addition to baseline models, exploratory experiments with **deep learning architectures** and transformer-based approaches were conducted to assess their ability to model non-linear and

high-dimensional interactions. While these models showed potential improvements in representational power, the hybrid approach was selected for deployment due to its balance between performance, interpretability, and computational efficiency.

The system is operationalized using a **FastAPI-based microservice architecture**, enabling scalable and low-latency inference. A web-based frontend interface is developed to facilitate user interaction, allowing for real-time prediction, parameter manipulation, and scenario simulation. This integration transforms the predictive models into a practical decision-support system for retail stakeholders.

From a business perspective, the system provides actionable insights into demand drivers, highlighting the influence of pricing strategies, marketing spend, and customer engagement on sales outcomes. It also identifies operational risks such as supplier delays and limited shelf life, enabling proactive mitigation strategies. The inclusion of simulation capabilities further enhances decision-making by allowing users to evaluate hypothetical scenarios and optimize inventory strategies accordingly.

In summary, this work demonstrates the effective application of machine learning in retail analytics through:

- A hybrid feature engineering framework integrating structured and contextual data
- Multi-task predictive modeling for classification and regression
- Deployment of scalable, real-time inference systems
- Translation of predictive outputs into actionable business insights

The proposed solution provides a robust and extensible foundation for intelligent retail decision systems and highlights the potential of AI-driven approaches in improving inventory efficiency, reducing operational risk, and enhancing overall business performance.

List of Symbols & Abbreviations

Abbreviation / Symbol	Full Form / Description
API	Application Programming Interface – used to expose model predictions and system functionalities via web services
ASGI	Asynchronous Server Gateway Interface – protocol used by Uvicorn to serve FastAPI applications
BERT	Bidirectional Encoder Representations from Transformers – deep learning model used for contextual text embeddings
BoW	Bag of Words – text representation method based on word frequency counts
CSV	Comma-Separated Values – standard format used for storing tabular datasets
DL	Deep Learning – neural-network-based modeling approach used for classification experiments
EDA	Exploratory Data Analysis – statistical and visual analysis used to understand dataset structure and relationships
F1 Score	Harmonic mean of precision and recall – used to evaluate classification performance
FastAPI	High-performance Python web framework used to deploy the retail intelligence system
IQR	Interquartile Range – statistical measure used for distribution spread and outlier detection
KNN	K-Nearest Neighbors – algorithm used for imputing missing values in preprocessing
MAE	Mean Absolute Error – average magnitude of prediction errors in regression
ML	Machine Learning – data-driven modeling techniques used for prediction and classification
PCA	Principal Component Analysis – dimensionality reduction technique used for latent structure analysis
RMSE	Root Mean Squared Error – measures standard deviation of prediction errors
R^2	Coefficient of Determination – indicates proportion of variance explained by regression model
TF-IDF	Term Frequency–Inverse Document Frequency – weighted text representation method
UI	User Interface – frontend dashboard used for system demonstration
Uvicorn	ASGI server used to run FastAPI applications
RF	Random Forest – ensemble learning algorithm used for feature importance analysis
Ridge Regression	Regularized linear regression model used for demand prediction

Logistic Regression	Classification model used for stocking-priority prediction
Transformer	Deep learning architecture used for advanced text modeling (Hugging Face benchmark)
NLP	Natural Language Processing – techniques used for handling product text data
SKU	Stock Keeping Unit – unique identifier for retail products
TF	Term Frequency – frequency of a term in a document (used in TF-IDF)
Feature Engineering	Process of transforming raw data into meaningful model inputs
Target Leakage	Occurs when future or outcome-derived information is used in training models
Profit Density	Ratio of gross profit to shelf capacity – key business metric for prioritization
Fill Rate	Percentage of demand fulfilled from available stock – indicator of supply reliability
Shelf Life	Duration for which a product remains sellable before expiration
Supplier Delay	Time lag in product replenishment – operational risk factor
Footfall Index	Proxy measure for customer traffic in a retail store
Discount %	Percentage reduction in product price relative to base price
Lead Time	Time between placing and receiving inventory orders
Hybrid Model	Model combining structured numerical features with text representations
Scenario Simulation	Process of evaluating model outputs under different hypothetical business conditions
Recommendation Engine	System component that ranks products based on business objectives
Stocking Priority Class	Target variable categorizing items into Low, Medium, or High priority
Daily Units Sold	Regression target representing product demand per day

Table of Contents

1. Problem statement 14

1.1 Problem Description 14

1.2 Formal Problem Definition 14

1.3 Dataset Context 15

1.4 Scope of the System 15

2. Objectives 16

2.1 Primary objectives 16

2.2 Secondary objectives 16

3. Previous Work in Retail Demand Forecasting and Inventory Optimization 17

3.1 Retail Demand Forecasting Models: 17

3.2 Machine Learning in Retail Analytics: 17

3.3 Deep Learning and Hybrid Approaches: 17

3.4 Inventory Optimization and Decision-Support Systems: 18

3.5 Research Gap 18

4. Detailed Plan of Work 19

5. Resources Required for the Project 21

5.1 Human Resources 21

5.2 Hardware Resources 21

5.3 Software & Development Environment 21

5.4 Execution & Deployment Environment 25

6. Methodology 25

6.1 Overview 25

6.2 Data Preparation and Preprocessing 25

6.3 Exploratory Data Analysis (EDA) 26

6.4 Feature Engineering 26

6.5 Target Engineering *27*

6.6 Model Development *28*

6.7 Model Selection *28*

6.8 Recommendation and Simulation Logic *29*

6.9 Deployment Architecture *29*

7. Potential Data Challenges and Project Risks 30

7.1 Data Quality Challenges *30*

7.2 Project Risks *31*

8. Dataset Overview 33

8.1 Dataset scope *33*

8.2 Feature groups *34*

8.3 Targets used in the project *34*

8.4 Data quality and preparation *35*

9. Exploratory Data Analysis 35

9.1 Distribution of the demand target *35*

9.2 Correlation analysis *37*

9.3 Feature importance from a demand perspective *38*

9.4 PCA and latent structure *39*

9.5 Temporal demand patterns *40*

9.6 Category and store-level segmentation *42*

9.7 EDA summary *43*

10. Feature Engineering 44

10.1 Business-oriented target engineering *44*

10.2 Numerical feature engineering *44*

10.3 Text feature engineering *46*

10.4 Scaling and interpretability *46*

10.5 Leakage control *46*

11. Model Development **47**

11.1 Week 3 regression baselines *47*

11.2 Final operational sales model *48*

11.3 Stocking-priority classification models *48*

11.4 Custom deep learning model *49*

11.5 Hugging Face transformer benchmark *49*

11.6 Final deployed model selection *49*

12. Model Evaluation **50**

12.1 Operational classification experiments *50*

12.2 Operational sales regression experiments *53*

12.3 Week 4 model-lab comparison *54*

12.4 Reconciling the two evaluation views *55*

13. System Architecture **55**

13.1 Layered architecture *58*

13.2 API design *58*

13.3 Frontend architecture *59*

13.4 Runtime resilience *59*

14. Results and Business Insights **60**

14.1 Category-level stocking-priority patterns *60*

14.2 Profit density and commercial efficiency *61*

14.3 Operational risk signals *61*

14.4 Interpretable business drivers *61*

14.5 Business interpretation of the integrated system *62*

15. Conclusion **63**

16. Future Work **64**

16.1 Methodological improvements *64*

16.2 Modeling extensions *64*

16.3 System and product extensions *64*

17. Bibliography / References **65**

18. Duly Completed Checklist **67**

1. Problem statement

1.1 Problem Description

Retail inventory management is inherently a multi-objective decision problem. While demand forecasting has traditionally been the central analytical task, real-world stocking decisions depend on a broader set of commercial and operational factors. A product with high expected demand may still be unsuitable for aggressive stocking if it has low profit margins, short shelf life, high storage requirements, or unreliable supplier performance. Conversely, products with moderate demand may be strategically prioritized due to higher profit density, longer shelf life, or stable replenishment cycles.

This complexity highlights a critical gap between academic demand forecasting models and practical retail decision-making. Conventional approaches often optimize for predictive accuracy alone, without incorporating operational feasibility and business constraints. As a result, there is a need for a more holistic framework that integrates demand signals with profitability, shelf efficiency, perishability, and supply-chain reliability.

1.2 Formal Problem Definition

The central research question addressed in this project is:

How can a retailer determine which products should be assigned low, medium, or high stocking priority while balancing demand, profitability, shelf efficiency, perishability, and supply-chain reliability?

This formulation extends beyond traditional regression-based forecasting and reframes the task as a decision-support problem. It requires not only estimating future demand but also translating multiple business factors into actionable stocking recommendations.

1.3 Dataset Context

The problem is addressed using a structured retail dataset comprising approximately **2,000 observations and 64 variables**, capturing both commercial and operational aspects of retail performance. The dataset includes:

- product and category metadata (e.g., category, brand, sub-category)
- pricing and promotion variables (e.g., base price, current price, discount percentage)
- customer engagement and traffic indicators (e.g., footfall index, website visits, loyalty usage)
- supply-chain and inventory attributes (e.g., shelf capacity, shelf life, fill rate, supplier delay)
- contextual variables (e.g., temporal indicators, weather, and seasonal signals)

This diverse feature space enables the modeling of both demand dynamics and operational constraints, making it suitable for developing a comprehensive inventory intelligence system.

1.4 Scope of the System

To address the problem, the project develops an integrated retail intelligence system composed of four interconnected stages:

- **Demand Estimation** – Predicting expected daily unit sales for each product
- **Stocking Priority Classification** – Assigning products to Low, Medium, or High priority categories based on business-oriented criteria
- **Decision Translation** – Converting model outputs into actionable recommendations, including risk indicators and business insights
- **System Deployment** – Exposing the functionality through a FastAPI-based application and an interactive dashboard for demonstration and decision support

This multi-stage approach ensures that the system moves beyond isolated prediction tasks and provides a practical, interpretable, and deployable solution aligned with real-world retail decision-making.

2. Objectives

The project is guided by a set of integrated technical and business objectives, reflecting the broader goal of developing a deployable retail decision-support system rather than a standalone predictive model.

2.1 Primary objectives

1. **To analyze the retail dataset** to understand demand distribution, pricing dynamics, customer traffic patterns, and operational factors influencing inventory outcomes.
2. **To perform feature engineering** across numerical, temporal, interaction, and text-based domains while ensuring strict control of target leakage.
3. **To construct a business-oriented target variable** for stocking priority that captures profitability, shelf efficiency, and operational reliability beyond raw sales volume.
4. **To develop and evaluate multiple machine learning models** for both regression and classification tasks, including classical algorithms, hybrid text–numeric models, and deep learning approaches.
5. **To select deployment-ready models** based on a balanced assessment of predictive performance, interpretability, computational efficiency, and robustness.
6. **To design and implement a FastAPI-based system** enabling real-time inference and integration of decision-support functionalities.
7. **To develop an interactive dashboard** that communicates model outputs and system insights effectively to non-technical stakeholders in a demonstration setting.

2.2 Secondary objectives

1. **To ensure experimental transparency** by documenting all explored models and methodologies, including those not selected for deployment.
2. **To translate model outputs into actionable business insights**, including risk indicators, profit density measures, and scenario-based comparisons.

3. **To maintain academic rigor** by explicitly addressing issues such as target leakage, model comparability, and trade-offs in model selection.

3. Previous Work in Retail Demand Forecasting and Inventory Optimization

3.1 Retail Demand Forecasting Models:

- **Fildes et al. (2019):** Showed that traditional statistical models (ARIMA, exponential smoothing) perform well for stable demand but struggle with multi-factor retail environments
- **Carbonneau et al. (2008):** Demonstrated that machine learning models outperform classical methods when incorporating pricing and promotional variables
- **Makridakis et al. (2020):** M4 competition results highlighted limitations of purely statistical forecasting in complex, real-world datasets

3.2 Machine Learning in Retail Analytics:

- **Hübner et al. (2016):** Identified pricing, promotions, and store traffic as key drivers of retail demand
- **Zhang et al. (2019):** Reported Random Forest and Gradient Boosting models achieving strong performance in retail demand prediction tasks
- **Bandara et al. (2020):** Showed that feature-rich machine learning models significantly improve forecasting accuracy over baseline approaches

3.3 Deep Learning and Hybrid Approaches:

- **Hochreiter & Schmidhuber (1997):** Introduced LSTM networks for sequential data modeling in demand forecasting
- **Vaswani et al. (2017):** Transformer models demonstrated strong performance in capturing contextual relationships in structured and text data

- **Salinas et al. (2020):** DeepAR and hybrid architectures improved forecasting performance in large-scale retail datasets

3.4 Inventory Optimization and Decision-Support Systems:

- **Silver et al. (1998):** Classical inventory models focused on cost minimization and replenishment policies
- **Choi et al. (2018):** Highlighted the importance of integrating demand forecasting with inventory decisions
- **Recent studies (2022–2024):** Emphasize forecast-to-decision pipelines combining prediction, risk assessment, and recommendation systems

3.5 Research Gap

Despite advancements in forecasting and machine learning, existing approaches primarily focus on **predicting demand rather than supporting inventory decisions**. There is limited work that integrates:

- demand forecasting with **profitability and operational constraints**
- structured features with **semantic product information (hybrid modeling)**
- predictive models with **deployable decision-support systems (API + dashboard)**

This project addresses these gaps by developing a **retail intelligence system** that combines demand prediction, stocking-priority classification, and business-oriented recommendations within a deployable framework.

4. Detailed Plan of Work

Task	Week 1	Week 2	Week 3	Week 4	Week 5	Week 6	Week 7	Week 8
Data Acquisition & Preparation								
Exploratory Data Analysis								
Feature Engineering & Baselines								
Advanced Modelling								
Model Selection & Integration								
API Development (FastAPI)								
Evaluation & Business Insights								
Dashboard & Final Submission								

Table 1: Plan of Work

Week	Phase	Key Activities	Deliverables
Week 1	Data Acquisition & Preparation	- Load and validate dataset - Handle missing values and encoding - Normalize features - Initial EDA and correlation analysis - Identify targets	- Cleaned dataset - EDA plots (distribution, correlation) - Data preprocessing pipeline
Week 2	Exploratory Data Analysis	- Demand distribution analysis - Feature correlation study - Random Forest feature importance - analysis - Temporal & category insights	- EDA report with visualizations - Feature importance charts - Insights summary
Week 3	Feature Engineering & Baselines	- Numerical and temporal feature engineering - Text feature creation (TF-IDF) - Train baseline regression models - Evaluate metrics - Identify leakage issues	- Feature-engineered dataset - Baseline model results - Leakage analysis report
Week 4	Advanced Modelling	- Create stocking priority target - Train classification models (logistic, hybrid) - Implement deep learning model - Transformer benchmark - Model comparison	- Classification models - Evaluation metrics (Accuracy, F1) - Model comparison report
Week 5	Model Selection & Integration	- Select final models (Hybrid Logistic + Ridge) - Build deployment-ready pipeline - Serialize models	- Final trained models (joblib/pickle) - Recommendation module

		- Implement recommendation engine - Scenario simulation logic	- Simulation engine
Week 6	API Development (FastAPI)	- Develop REST APIs - Create input schemas (Pydantic) - Implement endpoints - Deploy locally with Uvicorn - Test via Postman	- Functional API endpoints - API documentation - Sample request/response JSON
Week 7	Evaluation & Business Insights	- Evaluate deployed models - Analyse feature importance - Generate business insights - Run scenario simulations - Validate outputs	- Model evaluation report - Business insights report - Scenario analysis results
Week 8	Dashboard & Final Submission	- Build interactive dashboard - Integrate frontend with API - Add visualizations - Finalize report & slides - Prepare for viva	- Interactive dashboard - Final report document - Presentation slides

5. Resources Required for the Project

5.1 Human Resources

This project was developed as part of the BITS Pilani PGCP AIML Capstone under academic supervision.

- **Mentor:** *SUDARSHAN S. DESHMUKH)*
- **Student:**
    - **Sajal Debnath (2024AIML521)**
    - **Antony Sudharsan ( )**
    - **Kriti Sharma ( )**
    - **Ajay ()**

The project was implemented independently, covering the full lifecycle including data analysis, feature engineering, model development, evaluation, API deployment, and frontend integration.

5.2 Hardware Resources

The project was developed and executed on a personal laptop with the following configuration:

- **Model:** MacBook Pro (14-inch, 2021)
- **Chip:** Apple M1 Max
- **Memory:** 64 GB RAM
- **Operating System:** macOS Tahoe 26.4.1

The system provided sufficient computational capability for training machine learning models, running hybrid pipelines, and supporting local API deployment without the need for external GPU or cloud infrastructure.

5.3 Software & Development Environment

The project was implemented using Python and a modern data science and deployment stack.

Core Programming Language

- Python 3.x

Machine Learning & Modelling Libraries

- **Scikit-learn**

  Used for:

  - Logistic Regression (Stocking Priority Classification Model)
  - Ridge Regression (Sales Prediction Model)
  - Pipeline construction and preprocessing

- **XGBoost (Experimental)**

  Used for benchmarking model performance against ensemble methods

- **TensorFlow / Keras (Research Exploration)**

  Used to experiment with deep learning models for classification tasks

- **Hugging Face Transformers (Experimental)**

  Used for:

  - Text representation of product metadata
  - Exploring hybrid text + numeric modeling approaches

Data Processing & Feature Engineering

- **Pandas & NumPy**

  Used extensively for:

  - Data cleaning and transformation
  - Feature engineering
  - Handling structured retail dataset

- **Custom Feature Engineering Pipeline**

  Developed to:

  - Create numerical, temporal, and interaction features
  - Build structured text templates for hybrid modeling
  - Ensure leakage-free transformations for deployment

Visualization

- **Matplotlib**

  Used for:

  - Distribution plots
  - Correlation analysis
  - Model evaluation (confusion matrix, feature importance)

Deployment & API Layer

- **FastAPI**

  Used to build RESTful APIs for:

  - Stocking priority prediction
  - Sales prediction
  - Model comparison and scenario simulation

- **Uvicorn**

  ASGI server used to run the FastAPI application

- **Pydantic**

  Used for:

    - Input validation
    - Schema enforcement

Frontend & Interface

- **HTML + JavaScript (Custom UI)**

  Developed to:

    - Replace Swagger interface for demonstration
    - Provide structured input forms
    - Display predictions, probabilities, and insights clearly

Development Tools

- **Jupyter Notebook**

  Used for:

    - Exploratory Data Analysis
    - Feature engineering experiments
    - Model prototyping

- **Visual Studio Code (VS Code)**

  Used for:

    - Backend development
    - API integration
    - Project structuring

Environment & Dependency Management

- **pip + venv**

  Used for dependency management and environment isolation

5.4 Execution & Deployment Environment

- **Execution Mode:** Local system deployment
- **API Hosting:** FastAPI running on localhost
- **Frontend Access:** Browser-based UI via /ui endpoint

The system follows a modular architecture separating data processing, model inference, and presentation layers, enabling scalability and ease of future deployment to cloud environments.

All experiments and deployment components were implemented and validated within a unified local environment, ensuring reproducibility and consistency across the modeling and inference pipeline.

6. Methodology

6.1 Overview

The project follows an end-to-end machine learning pipeline designed to transform raw retail data into actionable decision-support outputs. The methodology integrates data preprocessing, feature engineering, predictive modelling, and deployment into a unified system capable of real-time inference.

The overall approach is structured into four key stages:

1. Data preparation and exploratory analysis
2. Feature engineering and target construction
3. Model development and evaluation
4. System integration and deployment

6.2 Data Preparation and Preprocessing

The initial phase involved loading and validating the Retail Master Dataset (v2), consisting of structured product-level attributes including pricing, customer engagement, supply chain variables, and external demand signals.

Key preprocessing steps included:

- Handling missing values using appropriate imputation strategies
- Standardizing numerical variables for model compatibility
- Encoding categorical variables where required
- Validating feature distributions and identifying outliers

Special attention was given to maintaining consistency between training and inference pipelines to ensure reliable deployment behaviour.

6.3 Exploratory Data Analysis (EDA)

Exploratory analysis was conducted to understand the relationships between key variables influencing retail demand and inventory decisions.

The analysis focused on:

- Distribution of demand (Daily Units Sold) across products and categories
- Impact of pricing, discounting, and competitor pricing on sales
- Role of customer engagement metrics such as footfall, website visits, and app traffic
- Correlation between operational variables such as fill rate, lead time, and supplier delay

Visualization techniques using Matplotlib were employed to generate distribution plots, correlation heatmaps, and category-level comparisons. Insights derived from EDA informed feature engineering and model selection decisions.

6.4 Feature Engineering

Feature engineering was a critical component of the methodology, designed to capture complex relationships within the retail environment.

The following feature categories were constructed:

Numerical Features

- Direct business variables such as price, marketing spend, and traffic indicators

- Interaction features capturing relationships between pricing and demand drivers

Temporal Features

- Derived features such as weekday and seasonal indicators to capture periodic demand variations

Text-Based Features

- Structured text templates combining product metadata (category, brand, promotion type, store type)
- Transformation using TF-IDF representations to enable hybrid modeling

Operational Features

- Supply chain indicators such as shelf life, fill rate, and supplier delay
- Used to capture inventory risk and replenishment constraints

All feature transformations were implemented in a leakage-free manner to ensure consistency between training and deployment environments.

6.5 Target Engineering

Two primary targets were defined to align with business objectives:

1. **Daily Units Sold (Regression Target)**
   - Continuous variable representing product-level demand
2. **Stocking Priority (Classification Target)**
   - Derived categorical variable (Low, Medium, High)
   - Constructed using business logic incorporating demand, profitability, and operational constraints

This dual-target approach enables the system to support both predictive analytics and decision-oriented classification.

6.6 Model Development

Multiple modelling approaches were explored to evaluate performance across different paradigms.

Regression Models (Sales Prediction)

- Linear Regression
- Ridge Regression (selected model)
- Random Forest Regressor (benchmark)

Evaluation metrics:

- Root Mean Squared Error (RMSE)
- Mean Absolute Error (MAE)
- R^2 Score

Classification Models (Stocking Priority)

- Logistic Regression (text-only, numeric-only, and hybrid variants)
- Random Forest Classifier (benchmark)
- Deep Learning Model (Keras-based, experimental)
- Transformer-based model (Hugging Face, exploratory)

Evaluation metrics:

- Accuracy
- F1-score
- Confusion Matrix

6.7 Model Selection

Final model selection was based on a balance of:

- Predictive performance
- Interpretability
- Computational efficiency
- Ease of deployment

The selected models were:

- **Hybrid Logistic Regression** for stocking priority classification
- **Ridge Regression** for sales prediction

These models provided strong performance while maintaining simplicity and robustness in deployment.

6.8 Recommendation and Simulation Logic

Beyond prediction, the system incorporates decision-support capabilities:

- **Recommendation Engine:**

Uses predicted outputs along with business heuristics to generate actionable insights such as high-priority items and risk flags

- **Caution Flags:**

Identifies operational risks such as low shelf life, supplier delays, and low fill rates

- **Scenario Simulation:**

Enables what-if analysis by modifying input variables (e.g., pricing, supply constraints) and observing model response

6.9 Deployment Architecture

The final system was deployed as a modular application using FastAPI.

Key components include:

- RESTful API endpoints for prediction and simulation
- Pydantic-based input validation
- Uvicorn server for local hosting
- Custom frontend interface for user interaction

The architecture separates:

1. Data processing layer
2. Model inference layer
3. Presentation layer

This modular design ensures scalability, maintainability, and ease of future extension.

7. Potential Data Challenges and Project Risks

7.1 Data Quality Challenges

1. **Missing Values and Data Incompleteness**
    - Several features such as marketing spend, competitor pricing, and external signals (e.g., Google Trends) may contain missing or sparse values due to inconsistent data collection.
    - Certain contextual fields (e.g., promotion type, festival indicators) may not be uniformly populated across all products.

    Mitigation:

    - Numerical features handled using mean/median imputation
    - Categorical features filled using default categories (e.g., “None”, “Unknown”)
    - Validation checks implemented during preprocessing pipeline
2. **Feature Noise and Proxy Variables**
    - Some features (e.g., footfall index, app traffic, website visits) act as indirect proxies for demand and may introduce noise.
    - External signals like Google Trends may not perfectly align with actual product-level demand.

    Mitigation:

    - Feature importance analysis used to filter weak predictors
    - Regularization (Ridge Regression) used to reduce overfitting
    - Model comparison across multiple algorithms
3. **Class Imbalance in Stocking Priority**
    - The derived target variable (Low, Medium, High priority) may be unevenly distributed depending on business rule thresholds.
    - Risk of model bias toward dominant classes.

    Mitigation:

- Use of class-weighted logistic regression
- Evaluation using F1-score in addition to accuracy
- Careful threshold design during target engineering

4. **Target Engineering Bias**
    - The stocking priority target is constructed using business rules rather than directly observed labels.
    - Risk of introducing subjective bias into the modeling process.

Mitigation:

- Transparent documentation of target construction logic
- Validation against demand and profitability indicators
- Sensitivity analysis through scenario simulation

5. **Limited Dataset Size**
    - Dataset consists of ~2,000 records, which is relatively small for training complex models such as deep learning or transformer-based architectures.
    - Limits generalization capability of high-capacity models.

Mitigation:

- Preference for simpler, interpretable models (Logistic Regression, Ridge)
- Cross-validation for robust evaluation
- Use of deep learning only for experimental benchmarking

7.2 Project Risks

1. **Model Generalization Risk**
    - Models trained on a limited dataset may not generalize well to new stores, regions, or product categories.

Mitigation:

- Use of regularized models
- Validation on hold-out data
- Modular design allowing retraining with new data

2. **Concept Drift**
    - Retail environments are dynamic, with frequent changes in pricing strategies, promotions, customer behavior, and supply chain conditions.

Mitigation:

- Design for periodic model retraining
- Monitoring of input feature distributions
- Extensible pipeline for incremental updates

3. **Overfitting in Hybrid and Deep Models**
    - Hybrid (text + numeric) and deep learning models may overfit due to limited dataset size.

Mitigation:

- Benchmarking against simpler models
- Selection of deployable models based on robustness, not just accuracy
- Avoidance of unnecessarily complex architectures in production

4. **Interpretability vs Performance Trade-off**
    - Complex models may provide marginally better accuracy but lack interpretability for business stakeholders.

Mitigation:

- Selection of Logistic Regression as final classification model
- Use of feature importance and probability outputs for explainability
- Inclusion of caution flags and business rules for transparency

5. **Deployment and Integration Risk**
    - Risk of mismatch between training pipeline and deployed inference pipeline.
    - Potential issues in API stability or input validation.

Mitigation:

- Use of consistent preprocessing pipeline in deployment
- Pydantic-based input validation in FastAPI
- Extensive local testing of endpoints

6. **User Adoption Risk**
    - Non-technical users may find model outputs difficult to interpret or trust.

    Mitigation:

    - Development of intuitive frontend dashboard
    - Clear display of predictions, probabilities, and explanations
    - Use of business-friendly outputs (priority labels, risk flags)
7. **Data Dependency Risk**
    - System performance depends heavily on availability of input features such as pricing, traffic, and supply chain variables.

    Mitigation:

    - Default handling for missing inputs
    - Flexible input schema in API
    - Graceful degradation of predictions

8. Dataset Overview

The final project uses a structured retail dataset stored in `**data/raw/master_retail_dataset_v2.csv**`. The dataset contains **2,000 observations** and **64 columns**, combining product descriptors, pricing variables, demand signals, store context, external conditions, and operational inventory variables.

8.1 Dataset scope

The dataset contains both commercial and operational views of retail performance:

- product descriptors such as category, sub-category, product name, brand, and brand tier
- pricing and promotion variables such as base price, current price, competitor price, discount percentage, promotion type, and marketing spend

- customer and traffic proxies such as footfall, app traffic, website visits, loyalty usage, repeat purchase rate, and basket size
- operational variables such as shelf capacity, shelf life, supplier delay, stock on hand, fill rate, and safety stock
- temporal and contextual variables such as day of week, month, holiday status, weather, and festival indicators

This breadth makes the dataset suitable for both demand forecasting and inventory intelligence tasks.

8.2 Feature groups

The precomputed insights artifacts organize the dataset into four major feature groups

Table 2: Feature Groups

Feature Group	Count	Description
Product and category metadata	8	Category, sub-category, product, brand, brand tier, festival, and store descriptors
Pricing and promotion	8	Base price, current price, discount, competitor price, promotion type, and marketing support
Shelf and supply-chain signals	8	Shelf capacity, shelf life, lead time, fill rate, delay, stock on hand, and safety stock context
Demand and engagement signals	12	Daily units sold, digital traffic, loyalty behavior, basket size, and demand proxies

8.3 Targets used in the project

The project uses two distinct targets.

Table 3: Targets Used in the Project

Target	Type	Purpose
Daily_Units_Sold	Regression	Used for demand forecasting and downstream revenue/profit estimation
Stocking_Priority_Class	Classification	Engineered target used to classify products into Low, Medium, or High stocking priority

The demand target is directly observed. The stocking-priority target is engineered from business logic, which is appropriate because real stocking priority is usually a managerial decision variable rather than a naturally labeled field in the source data.

8.4 Data quality and preparation

The notebooks and training scripts indicate that the dataset required light but important preparation:

1. binary flags were standardized to numeric indicators
2. selected numeric columns were coerced to numeric and imputed where necessary
3. text-like business columns were normalized and combined into structured text representations
4. date columns were converted into calendar-derived features where relevant

The final dataset is therefore rich enough to support both exploratory analytics and production-style model pipelines.

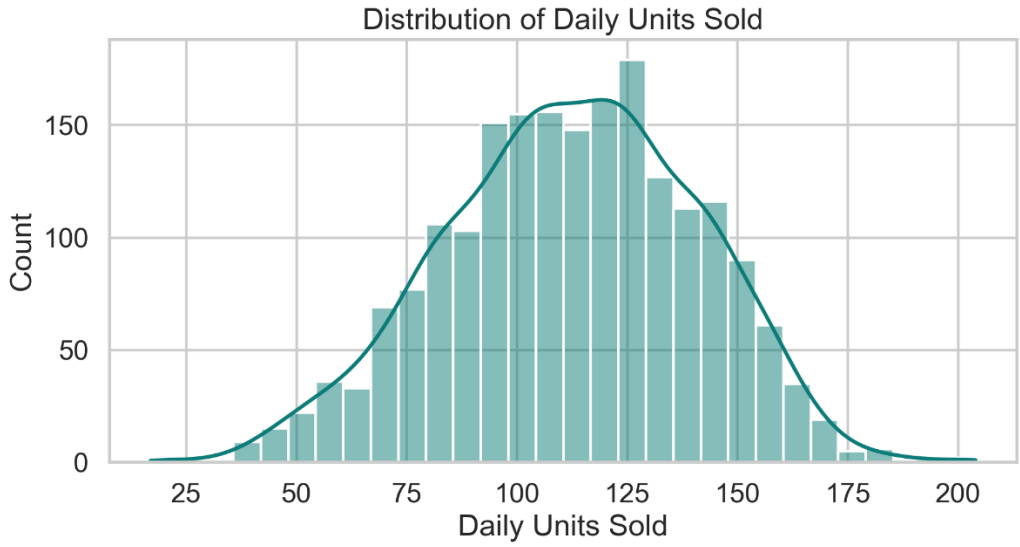
9. Exploratory Data Analysis

Exploratory data analysis established the statistical and business context for the later modeling stages. It also helped determine which variables were structurally informative, which variables were redundant, and which transformations were worth carrying into later experiments.

9.1 Distribution of the demand target

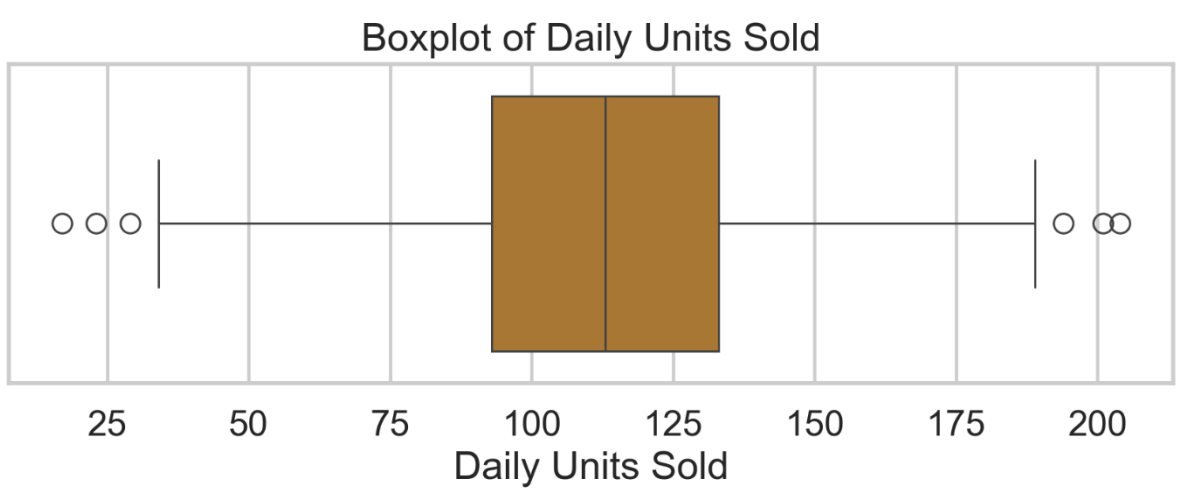
The first question was whether the regression target, *Daily_Units_Sold*, had a reasonable distribution for supervised learning.

Figure 1: Distribution of Daily Units Sold



The distribution is centered around a mean of **112.26 units** and a median of **113 units**, with a standard deviation of **28.58 units**. The interquartile range spans from **93** to **133** units. The closeness of the mean and median indicates that the target is not severely skewed, and the density curve suggests that the distribution is reasonably smooth rather than fragmented into disconnected modes. This is favorable for both linear and tree-based regressors.

Figure 2: Boxplot of Daily Units Sold

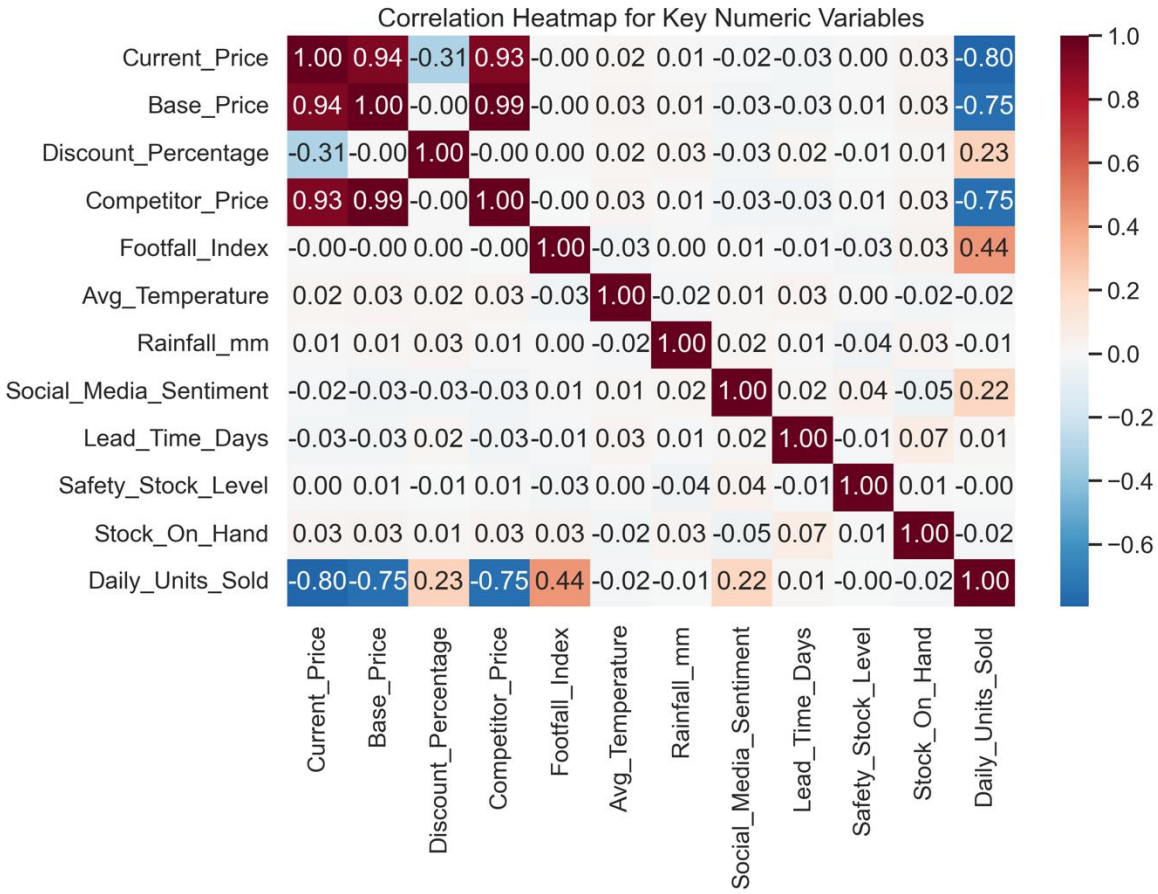


The boxplot confirms that the target contains some low and high extremes, but not in a way that suggests pathological outliers. The observed range extends from **17** to **204** units. That spread is wide enough to make forecasting meaningful yet compact enough that simple regression families remain viable. This finding justified the use of classical baselines such as linear, Ridge, and tree-based regressors in Week 3.

9.2 Correlation analysis

Correlation analysis was used to understand which continuous variables moved with demand and which variables likely captured overlapping information.

Figure 3: Correlation Heatmap for Key Numeric Values



Three insights stand out immediately. First, *Current_Price* has the strongest negative relationship with the target at approximately **-0.80**, which is consistent with basic price elasticity: higher selling prices generally suppress unit movement. Second, *Footfall_Index* has the strongest positive relationship with demand at approximately **0.44**, showing that customer traffic is a

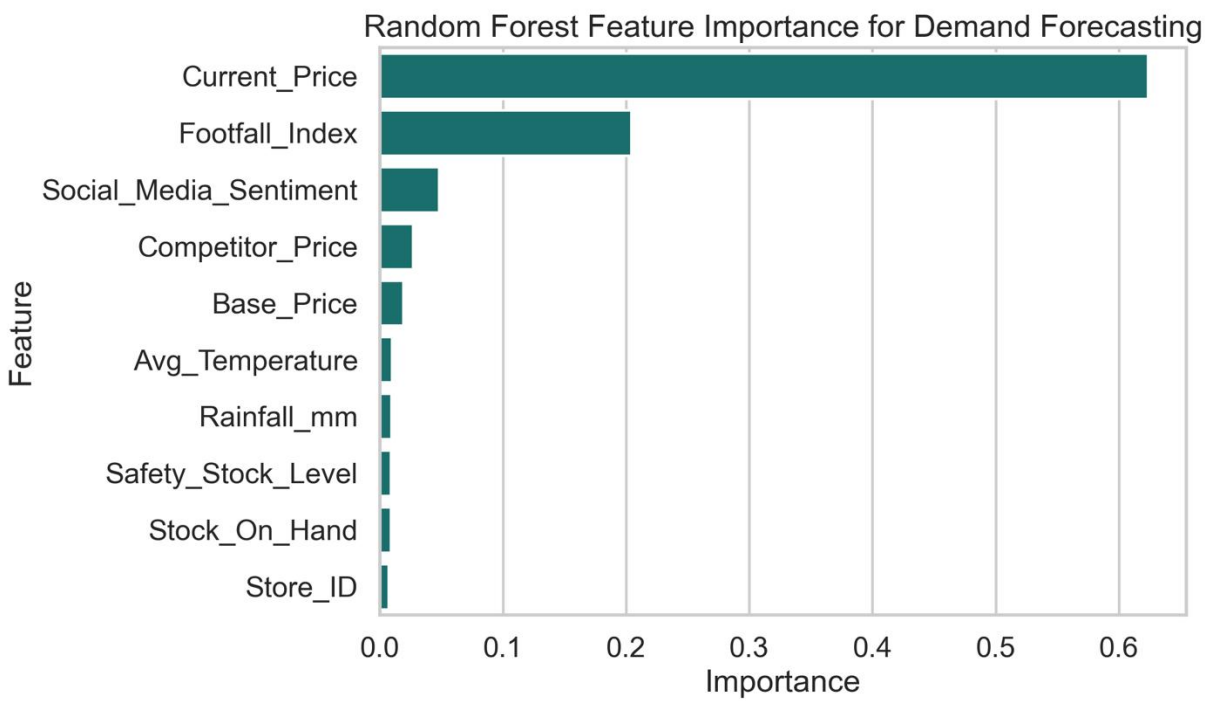
major commercial driver. Third, *Discount_Percentage* and *Social_Media_Sentiment* are both positively associated with demand, at roughly **0.23** and **0.22**, suggesting that both promotional pressure and consumer perception contribute to product movement.

The heatmap also reveals very high correlation among pricing variables such as *Base_Price, Current_Price*, and *Competitor_Price*. This is not surprising because these variables all describe related aspects of retail pricing. The implication for modeling is that multicollinearity must be handled carefully. Linear models can still be used, but their coefficients require interpretation with awareness of correlated predictors.

9.3 Feature importance from a demand perspective

To go beyond linear correlation, the EDA notebook estimated feature importance using a Random Forest regressor and then validated those findings with permutation importance.

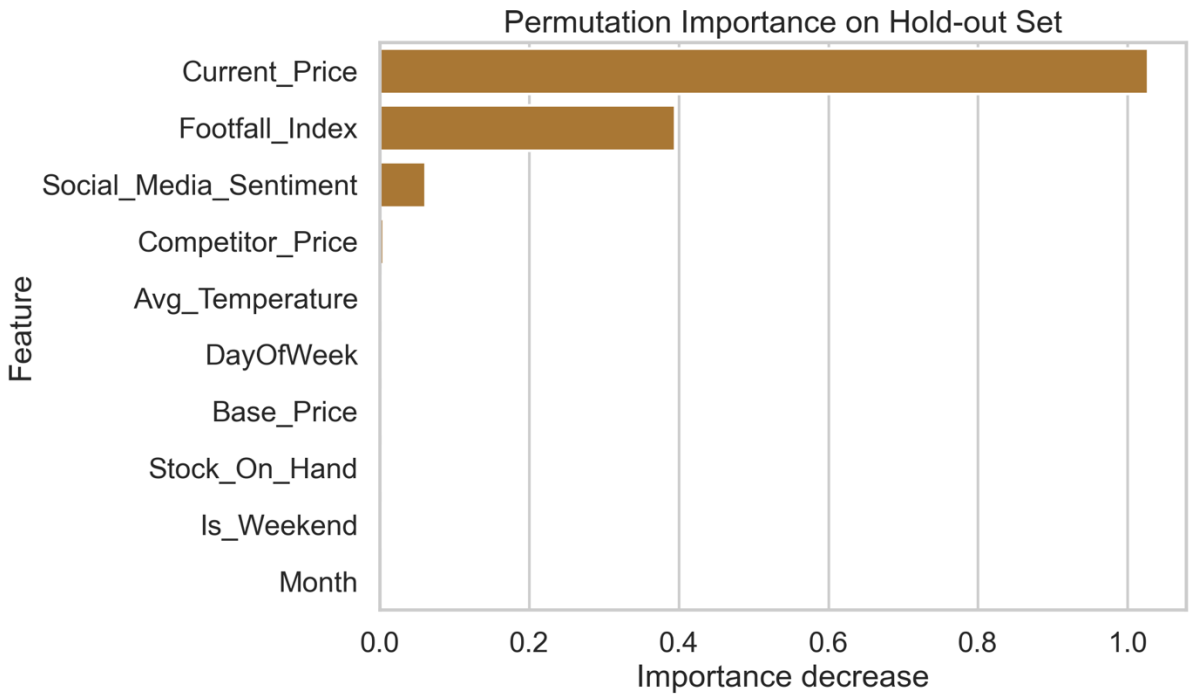
Figure 4: Random Forest Feature Importance for Demand Forecasting



The Random Forest importance ranking is sharply dominated by *Current_Price* (**0.6235**) and *Footfall_Index* (**0.2043**). *Social_Media_Sentiment* follows at **0.0482**, while all remaining

variables contribute much less. This indicates that the demand signal is concentrated in a few commercially meaningful drivers rather than being evenly spread across the full feature set.

Figure 5: Permutation Importance on Hold-out Set

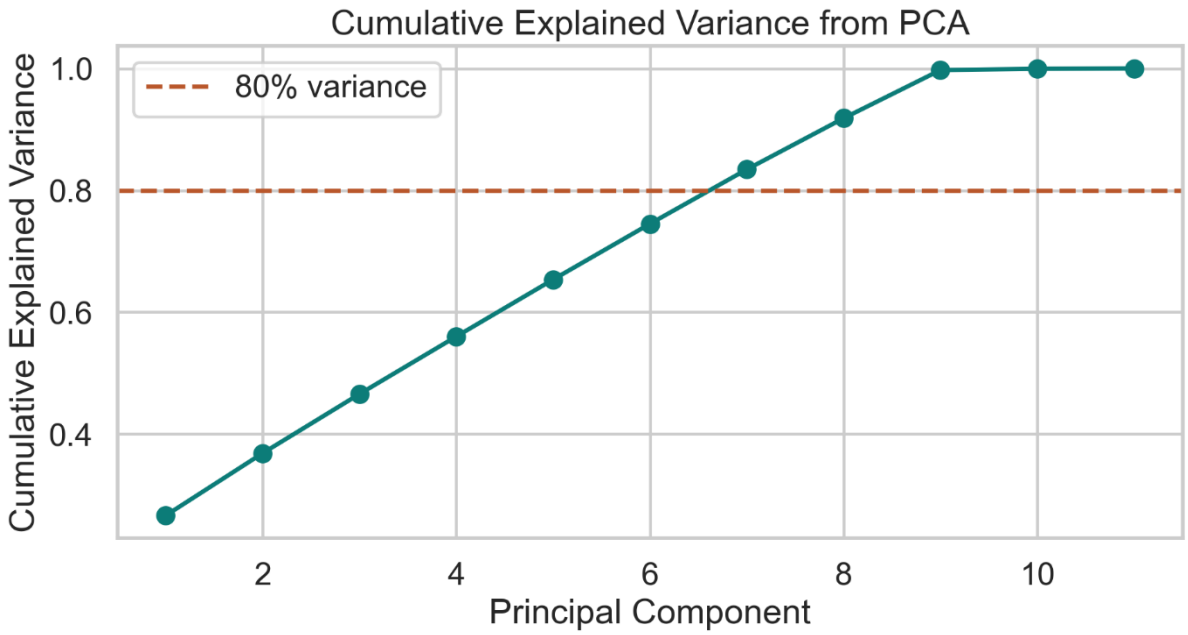


Permutation importance confirms the same conclusion. *Current_Price* remains the dominant feature with an importance decrease of roughly **1.027**, followed by *Footfall_Index* (**0.395**) and *Social_Media_Sentiment* (**0.061**). The consistency between the two importance methods is important. It shows that the signal is not an artifact of one model family. For the report, this supports a strong business interpretation: pricing discipline, store traffic, and customer perception are the primary drivers of retail demand in this dataset.

9.4 PCA and latent structure

Principal Component Analysis was used not as a production requirement, but as an exploratory device to understand whether broad latent factors existed across the numeric feature block.

Figure 6: Cumulative Explained Variance from PCA

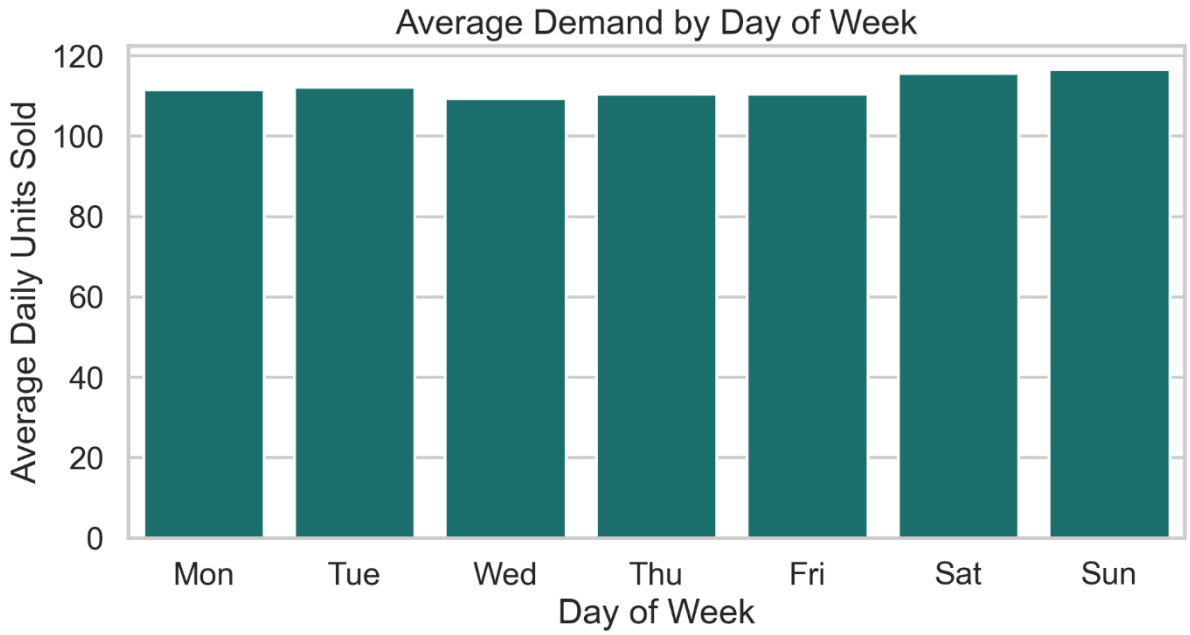


The first principal component explains about **26.61%** of the variance. The first five components together explain approximately **65.35%** of the total variance. This implies that the dataset does contain shared latent structure across pricing, traffic, and operational variables, but not so strongly that a tiny component set can replace the original features without information loss. In practical terms, PCA was useful for exploration and dimensionality studies, but not necessary for the final deployed models.

9.5 Temporal demand patterns

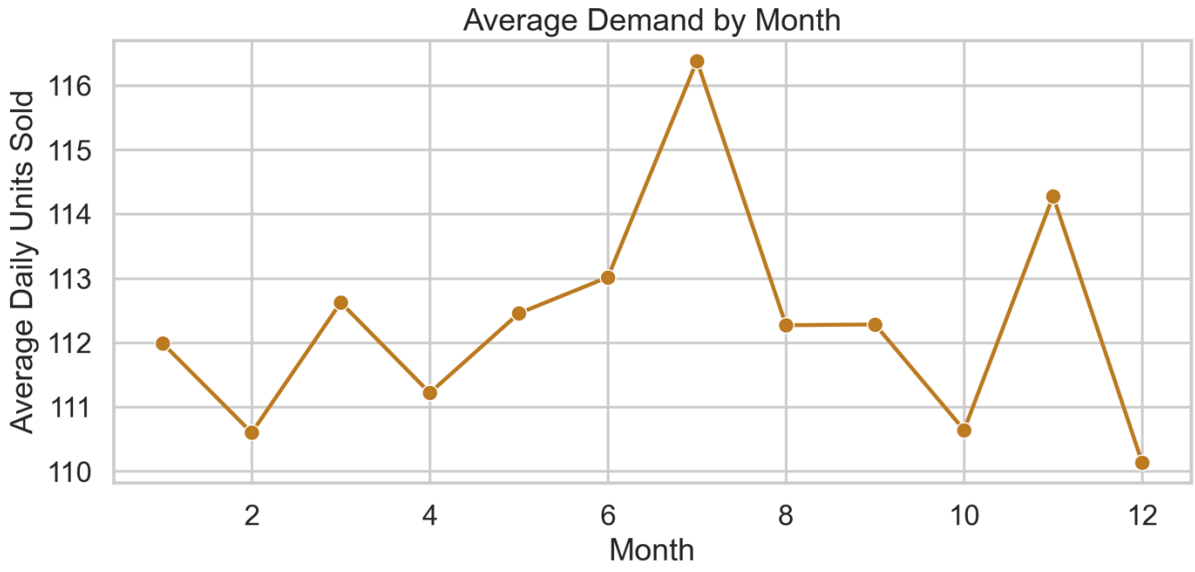
Although the deployed app is not a classical time-series forecaster, temporal analysis remains useful because retail demand is often shaped by weekdays, weekends, and month-level seasonality.

Figure 7: Average Demand by Day of Week



Demand is slightly lower mid-week and rises during the weekend. The lowest observed day-level average is around **109.30 units** on Wednesday, while demand peaks at roughly **115.61** on Saturday and **116.49** on Sunday. The pattern is not extreme, but it is meaningful enough to justify retaining weekday and weekend flags in exploratory modeling and feature engineering.

Figure 8: Average Demand by Month

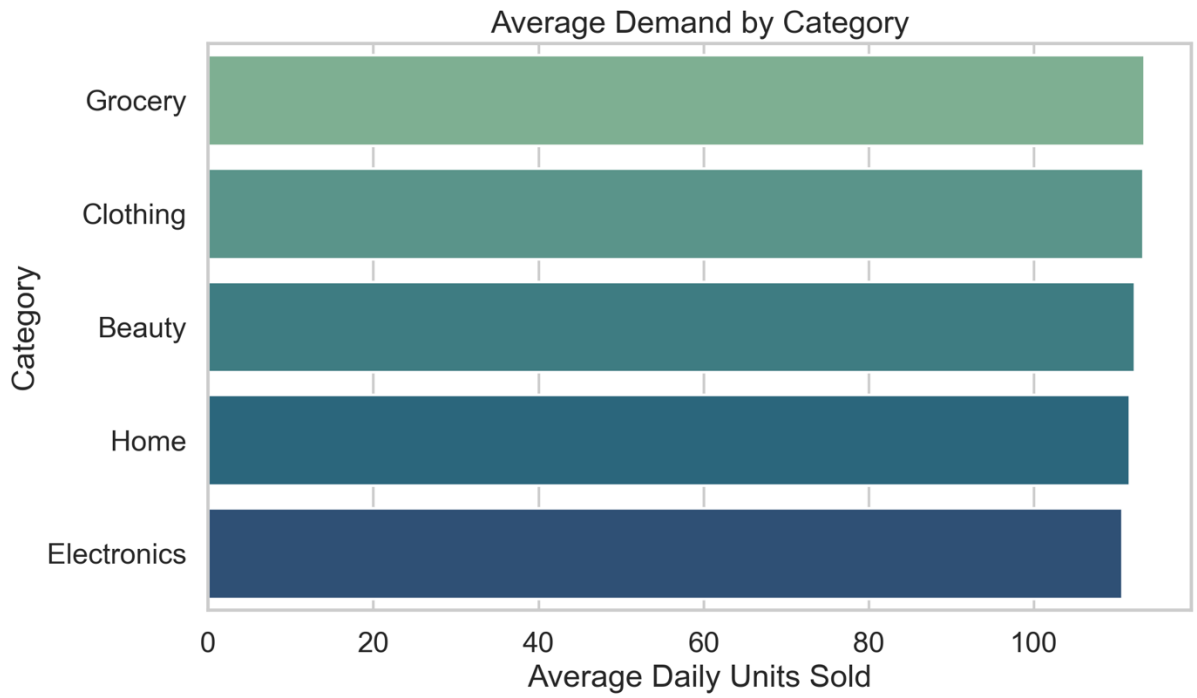


Monthly variation is present but moderate. The highest monthly average appears in **July (116.38 units)**, followed by **November (114.28 units)**, while the lower months include **December (110.13 units)** and **February (110.60 units)**. This suggests that demand seasonality exists, but it is not the sole driver of the outcome. It acts more as contextual variation than as a dominant pattern.

9.6 Category and store-level segmentation

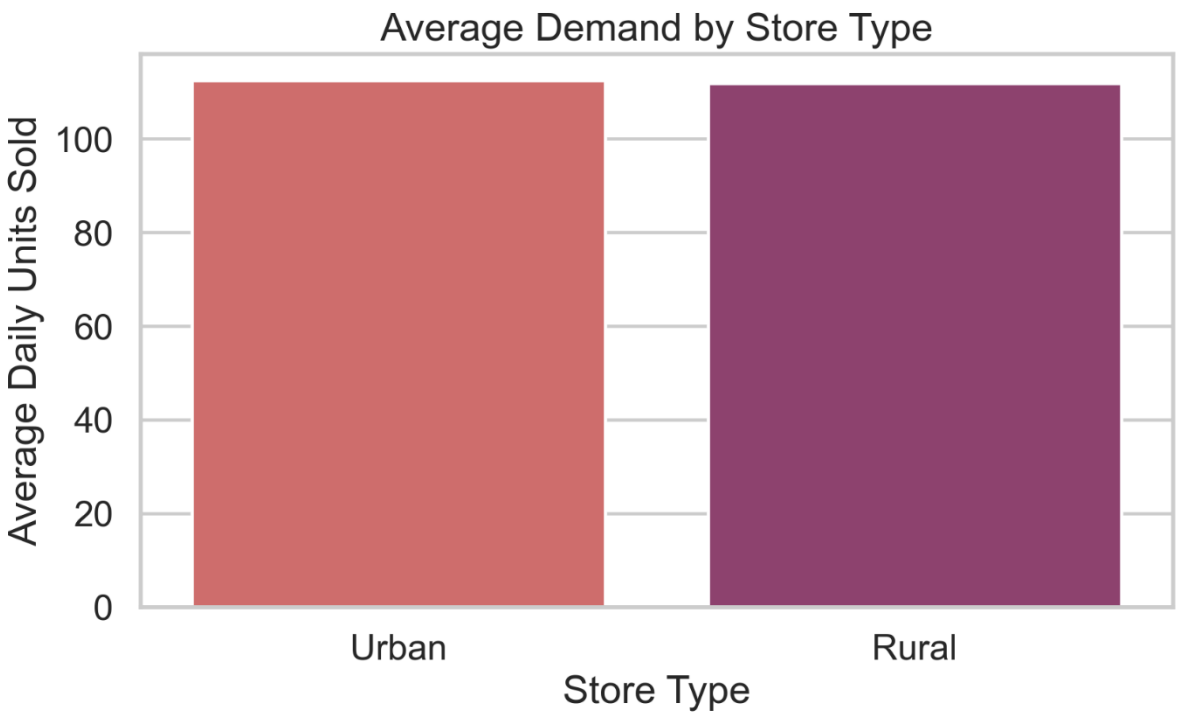
Category and store-level plots were used to understand whether the dataset exhibits strong structural differences across product classes or retail formats.

Figure 9: Average Demand by Category



The category-level averages are relatively close, but still informative. *Grocery* has the highest average demand at **113.40 units**, followed closely by *Clothing* (**113.26**) and *Beauty* (**112.24**). *Electronics* is lowest at **110.69**. The small spread indicates that demand alone does not sharply separate categories. This helps explain why a separate stocking-priority target is valuable: a category may have respectable demand but still be disadvantaged by shelf-life or supply constraints.

Figure 10: Average Demand by Store Type



Urban stores show a marginally higher average demand (**112.46 units**) than rural stores (**111.96 units**). The difference is modest, which suggests that store type is useful context rather than a decisive determinant. This finding is consistent with the eventual modeling choice: store-type context is retained, but it is not treated as a dominant standalone predictor.

9.7 EDA summary

The exploratory phase established five strong conclusions:

1. the regression target is well-behaved enough for supervised learning
2. price and footfall are the dominant demand drivers
3. pricing variables are highly collinear and must be interpreted carefully
4. temporal and category-level effects are present, but not overwhelmingly large
5. demand alone cannot explain stocking decisions, which motivates a richer business target

10. Feature Engineering

Feature engineering was the most conceptually important part of the project, because it connected raw retail variables to the final business questions being solved.

10.1 Business-oriented target engineering

The most important engineered construct in the project is the stocking-priority target itself. Rather than using sales volume alone, the project constructs a business-oriented score from profitability and operational reliability.

The scripts and Week 4 notebook define the following intermediate variables:

1. **Unit Margin** = *Current_Price - Unit_Cost*
2. **Gross Profit** = *Unit_Margin × Daily_Units_Sold*
3. **Profit Density** = *Gross_Profit / Shelf_Capacity*

These variables are then normalized and combined into a stocking-priority score using the following weights:

$$\begin{aligned} Score \ = \ & 0.45 \cdot ProfitDensity_n \\ & + 0.20 \cdot ShelfLife_n \\ & + 0.20 \cdot FillRate_n \\ & - 0.10 \cdot SupplierDelay_n \\ & - 0.05 \cdot BackorderFlag_n \end{aligned}$$

This score is divided into three quantile-based classes:

- Low
- Medium
- High

The resulting class distribution is almost perfectly balanced: **667 High**, **667 Low**, and **666 Medium**. This is helpful for classification training, but more importantly it produces a label that reflects realistic merchandising trade-offs rather than raw movement alone.

10.2 Numerical feature engineering

Week 3 feature-engineering notebooks explored a wide range of structured transformations:

1. calendar features such as weekday, month, holiday markers, and weekend flags
2. lag features for prior-demand context
3. rolling statistics for short-term demand behavior

4. price–promotion interaction features
5. store-level and product-level aggregation features
6. Customer and supply-chain derived features
7. standardization and PCA on numeric blocks

These transformations were academically valuable because they showed how demand behavior can be enriched through context. However, the final deployed system deliberately uses a narrower numeric feature block. The reason is architectural: live API inference often occurs for a single hypothetical product scenario, where historical lag windows or rolling demand histories may not be available. The deployment therefore favors features that can be supplied directly at inference time.

The final numeric features used in the operational models are:

- current price
- shelf capacity
- lead time
- marketing spends
- shelf life
- fill rate
- supplier delay
- discount percentage
- competitor price
- footfall index
- average temperature
- rainfall
- Google Trends current week
- Google Trends lags 1 week
- app traffic index
- website visits
- loyalty usage count
- repeat purchase rate
- average basket size

This selection creates a strong balance between explanatory richness and deployability.

10.3 Text feature engineering

Text representation was explored extensively in Week 3 and Week 4 because product semantics matter in retail. Category, product name, brand, and promotional information all contain useful business cues that are hard to capture with purely numeric models.

The notebooks evaluated four main text-representation strategies:

1. bag-of-words
2. TF-IDF
3. Word2Vec-style embeddings
4. BERT / transformer embeddings

The important insight from this exploration was that unstructured text should not simply be concatenated carelessly. The improved approach was a **column-aware text template**, which preserves the role of each token in business context. The deployed template is:

```
Category: ... | Product: ... | SubCategory: ... | Brand: ... | BrandTier: ... |
Promotion: ... | Festival: ... | StoreType: ...
```

This structure matters because it allows a lightweight TF-IDF + logistic pipeline to distinguish brand, category, and promotion semantics in a more meaningful way than a flat text string.

10.4 Scaling and interpretability

Standardization was used for the numeric features in the final deployed models. This serves two purposes:

1. it helps optimization in linear models such as logistic regression and Ridge regression
2. it makes coefficient magnitudes more comparable within a model family

This decision improves interpretability. In the final insight layer, coefficients can be discussed directly as business levers, such as the positive role of fill rate and shelf life or the negative role of supplier delay.

10.5 Leakage control

One of the strongest technical features of the final project is that leakage is explicitly acknowledged and controlled.

For the stocking-priority classifier, the following variables are excluded from the live feature set:

- `Daily_Units_Sold`
- `Unit_Margin`
- `Gross_Profit`
- `Profit_Density`
- `Stocking_Priority_Score`
- `Stocking_Priority_Class`

For the deployed sales regression model, post-outcome columns such as *Online_Sales_Units* and *In_Store_Sales_Units* are excluded. This is crucial because the Week 3 regression notebook demonstrated what happens when leakage is present: linear regression produced nearly perfect metrics because sales decomposition variables implicitly revealed the target. That result was analytically useful as a warning sign, but it is not appropriate for deployment.

The final production pipeline is therefore much more defensible than the early baseline experiments because it distinguishes exploratory experimentation from operational modeling.

11. Model Development

The modeling phase evolved in stages. Early work emphasized demand forecasting baselines and feature experimentation, while later work focused on the final retail decision-support architecture.

11.1 Week 3 regression baselines

The Week 3 notebook compared several baseline regressors on the engineered dataset.

Table 4: Regression Baselines

Model	MAE	RMSE	R^2
Linear Regression	0.0000	0.0000	1.0000
Ridge Regression	0.0441	0.0570	1.0000
Lasso Regression	0.1086	0.1342	1.0000
Random Forest Regressor	1.0511	1.9304	0.9955

At first glance these numbers appear extraordinary, but they must be interpreted carefully. The same notebook reports very high importance for *In_Store_Sales_Units* and *Online_Sales_Units*. Since those variables are decomposition-level views of the sales outcome, they effectively leak the target into the model. The near-perfect fit therefore reflects information leakage rather than genuine forecasting power.

This stage of the project was still valuable because it revealed exactly why a stricter feature policy was needed. The experience directly informed the final deployed sales model, which removed post-outcome variables and retained only inference-available business features.

11.2 Final operational sales model

The final deployed sales model is a **log-transformed Ridge regressor** implemented through *TransformedTargetRegressor* in the training script. The model receives the same column-aware text template used elsewhere, along with the leakage-free numeric business block. This model was chosen because:

1. Ridge regression is stable under multicollinearity, which is important for price-related variables
2. the log transformation helps manage positive demand targets with moderate spread
3. the model is lightweight, fast, and easy to serve in FastAPI
4. its outputs can be translated into revenue, profit, and shelf-utilization summaries for the frontend

11.3 Stocking-priority classification models

The project evaluated multiple logistic-regression variants:

1. text-only logistic regression
2. numeric-only logistic regression
3. hybrid logistic regression combining text and numeric features

The rationale for this family comparison was straightforward. A text-only model can test how much semantic product context matters. A numeric-only model can test the power of structured operational features. A hybrid model can determine whether combining both views produces a more balanced signal.

11.4 Custom deep learning model

The Week 4 notebook also implemented a custom deep learning classifier that combined text-style input with structured business context. The academic motivation was clear: neural architecture could potentially capture non-linear interactions among promotions, category semantics, supply variables, and commercial signals in ways that linear models cannot. However, the project also recognized the deployment trade-off. A custom deep model increases dependency complexity, tuning burden, runtime overhead, and explainability cost. These issues matter more in a capstone demo environment than in a purely research-oriented notebook.

11.5 Hugging Face transformer benchmark

The Hugging Face benchmark was included as a modern NLP comparison point. It tests whether richer language representations help when the business context is encoded as a structured text template.

This was academically useful because it establishes that the project explored contemporary NLP methods rather than relying only on classical pipelines. At the same time, the report must be honest: the transformer setup is text-only, so it underuses structured operational constraints such as fill rate, shelf life, and supplier delay. That mismatch is one reason it was not chosen for deployment.

11.6 Final deployed model selection

The final deployed application uses:

- **Hybrid Logistic Regression** for stocking-priority classification
- **Log-Ridge Sales Regression** for demand forecasting

These models were selected because they provide a good balance across four criteria:

1. predictive adequacy
2. business interpretability
3. deployment simplicity
4. runtime stability

This selection logic is important. The project does not claim that the simplest model is always numerically best model in every experiment. Instead, it argues that the chosen models provide the strongest practical balance for a robust academic demo and decision-support application.

12. Model Evaluation

Model evaluation in this project should be interpreted in two layers:

1. **operational experiments** that support the deployed app
2. **broader Week 4 comparisons** that document the full model-exploration story

These two views are complementary, but they are not numerically identical because they arise from different experiment workflows.

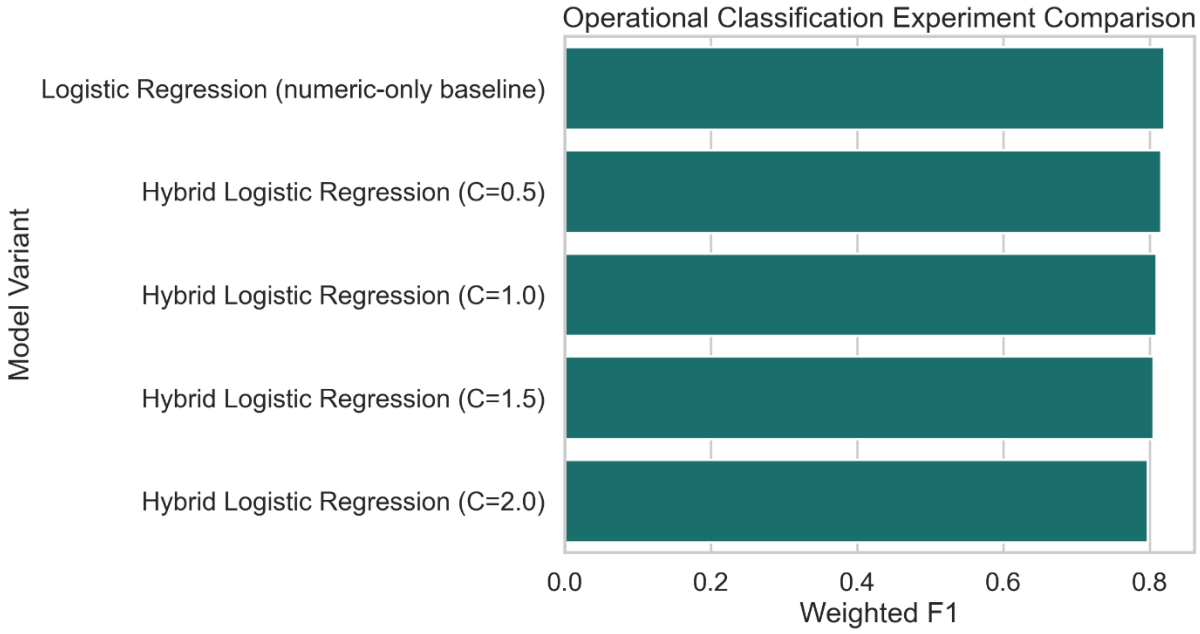
12.1 Operational classification experiments

The precomputed operational classification results compare a numeric-only logistic baseline against a small sweep of hybrid logistic models.

Table 5: Operational Classificational Experiments

Model Variant	Accuracy	Macro F1	Weighted F1
Logistic Regression (numeric-only baseline)	0.8200	0.8203	0.8204
Hybrid Logistic Regression (C=0.5)	0.8150	0.8154	0.8155
Hybrid Logistic Regression (C=1.0)	0.8100	0.8097	0.8098
Hybrid Logistic Regression (C=1.5)	0.8050	0.8051	0.8052
Hybrid Logistic Regression (C=2.0)	0.7975	0.7977	0.7978

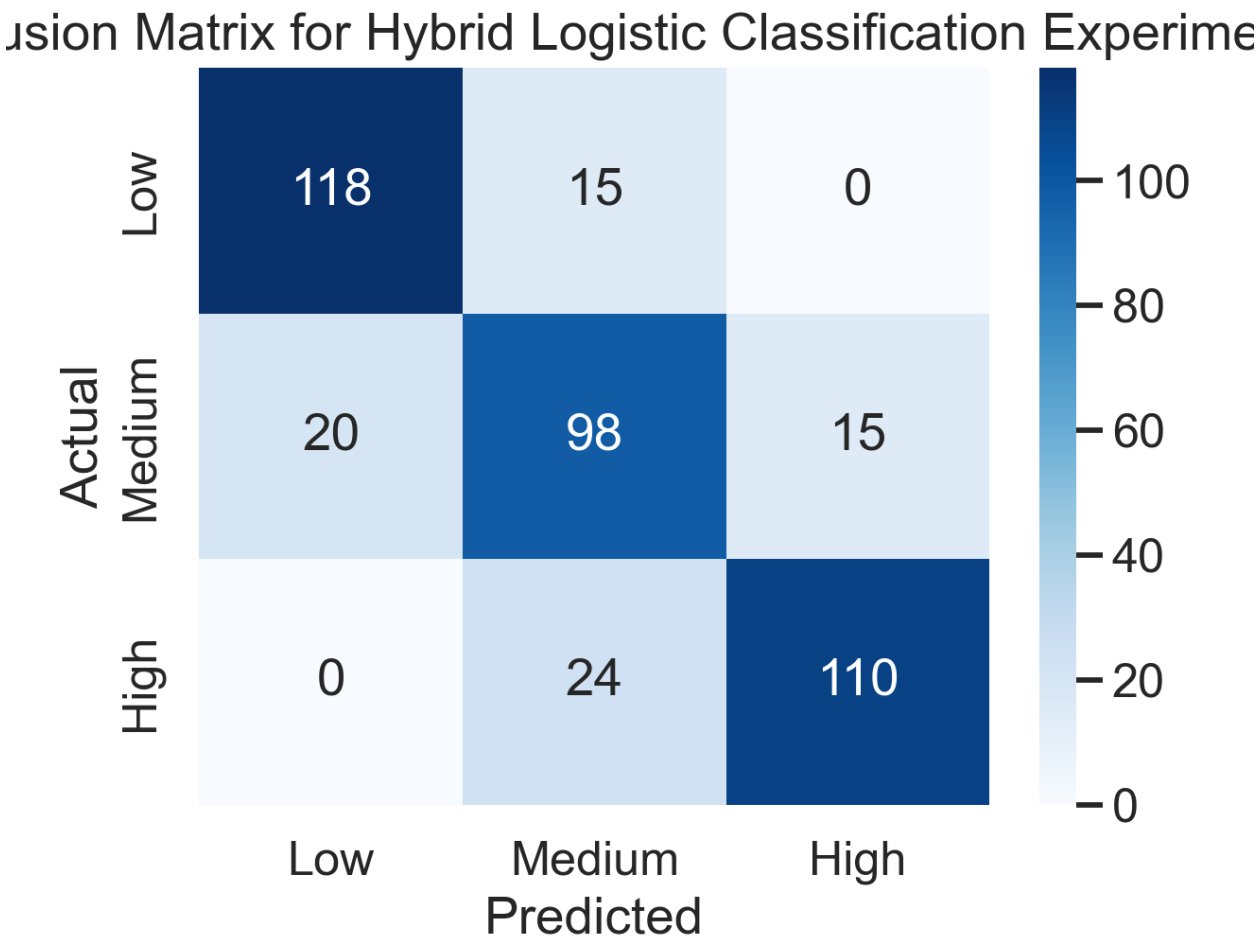
Figure 11: Operational Classification Experiment Comparison



The most important observation here is nuanced. In this particular operational sweep, the **numeric-only baseline slightly outperforms the hybrid sweep**. This implies that the engineered stocking-priority label is already heavily explained by structured business variables such as fill rate, shelf life, pricing, and supplier behavior. That is a reasonable result because the label itself was constructed from business quantities derived from those variables.

However, it would be incorrect to conclude from this one table that text is useless. The hybrid model remains valuable for deployment because product semantics, category context, brand cues, and promotion language are useful for live business scenarios and for the broader Week 4 comparison discussed later.

Figure 12: Confusion Matrix of Hybrid Logistic Classification Experiment



The confusion matrix shows that the **Medium** class is the most difficult to identify cleanly. Low and High are classified more confidently, while Medium is often confused with its neighboring classes. This is expected in a three-level merchandising problem because Medium typically represents the boundary zone where attractive and risky signals coexist.

12.2 Operational sales regression experiments

The deployed sales model family was evaluated using a Ridge alpha sweep.

Table 6: Sales Regression Experiment Comparison

Model Variant	MAE	RMSE	R^2
Ridge alpha=5.0	9.5546	11.9143	0.8193
Ridge alpha=2.0	9.6470	12.0334	0.8157
Ridge alpha=1.0	9.7058	12.1194	0.8130
Ridge alpha=0.5	9.7626	12.2001	0.8105

Figure 13: Sales Regression Experiment Comparison

These results are realistic and credible. RMSE stays in a narrow band of about **11.9 to 12.2 units**, while R^2 remains between **0.81 and 0.82**. This tells us three things. First, the sales task is learnable with a classical linear model. Second, the feature block captures meaningful commercial signal without relying on leakage. Third, more complex demand models are not strictly necessary for a stable deployed demo.

The deployed artifact uses the *alpha=1.0* variant for consistency and simplicity, while the artifact summary also records that the best offline RMSE in the sweep came from *alpha=5.0*.

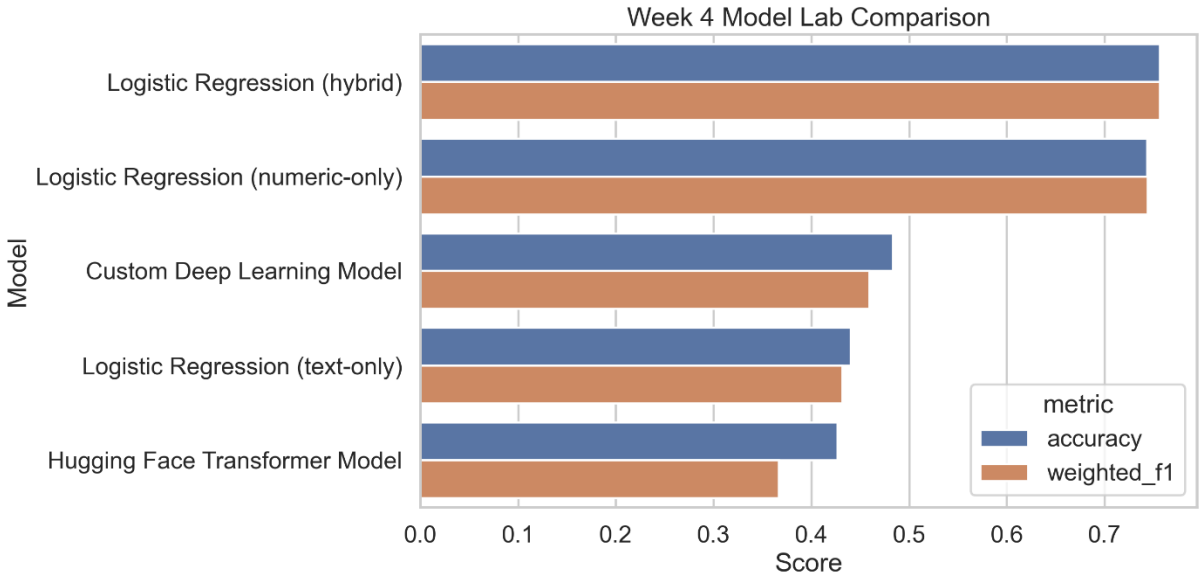
12.3 Week 4 model-lab comparison

The broader Week 4 benchmark compares the major classification families explored in the capstone.

Table 7: Model Lab Comparison

Model	Accuracy	Weighted F1	Runtime Status
Logistic Regression (hybrid)	0.7567	0.7563	Live compare and deployed family
Logistic Regression (numeric-only)	0.7433	0.7441	Live compare
Custom Deep Learning Model	0.4833	0.4589	Benchmark-only
Logistic Regression (text-only)	0.4400	0.4314	Live compare
Hugging Face Transformer Model	0.4267	0.3666	Benchmark-only

Figure 14: Model Lab Comparison



This benchmark is important because it answers a different research question from the operational sweep. In this broader comparison, the **hybrid logistic model performs best overall**, outperforming both the numeric-only and text-only logistic baselines and dramatically outperforming the text-only transformer benchmark. The deep learning model improves over text-only baselines, but it still falls well below the hybrid logistic model while being far more expensive to operate.

The result supports an academically mature conclusion: model sophistication alone does not guarantee better performance. In this retail context, **representation quality and feature alignment with the business problem matter more than architectural complexity**.

12.4 Reconciling the two evaluation views

At first glance, the operational classification sweep and the Week 4 model-lab benchmark may seem inconsistent because one shows the numeric-only baseline slightly ahead, while the other shows the hybrid model ahead. This is not an error; it is a consequence of comparing different experimental workflows:

1. the operational sweep is a narrow deployment-focused comparison against the engineered label
2. the model-lab benchmark is a broader representation study across multiple modeling families

The report should therefore avoid over-claiming. The correct interpretation is:

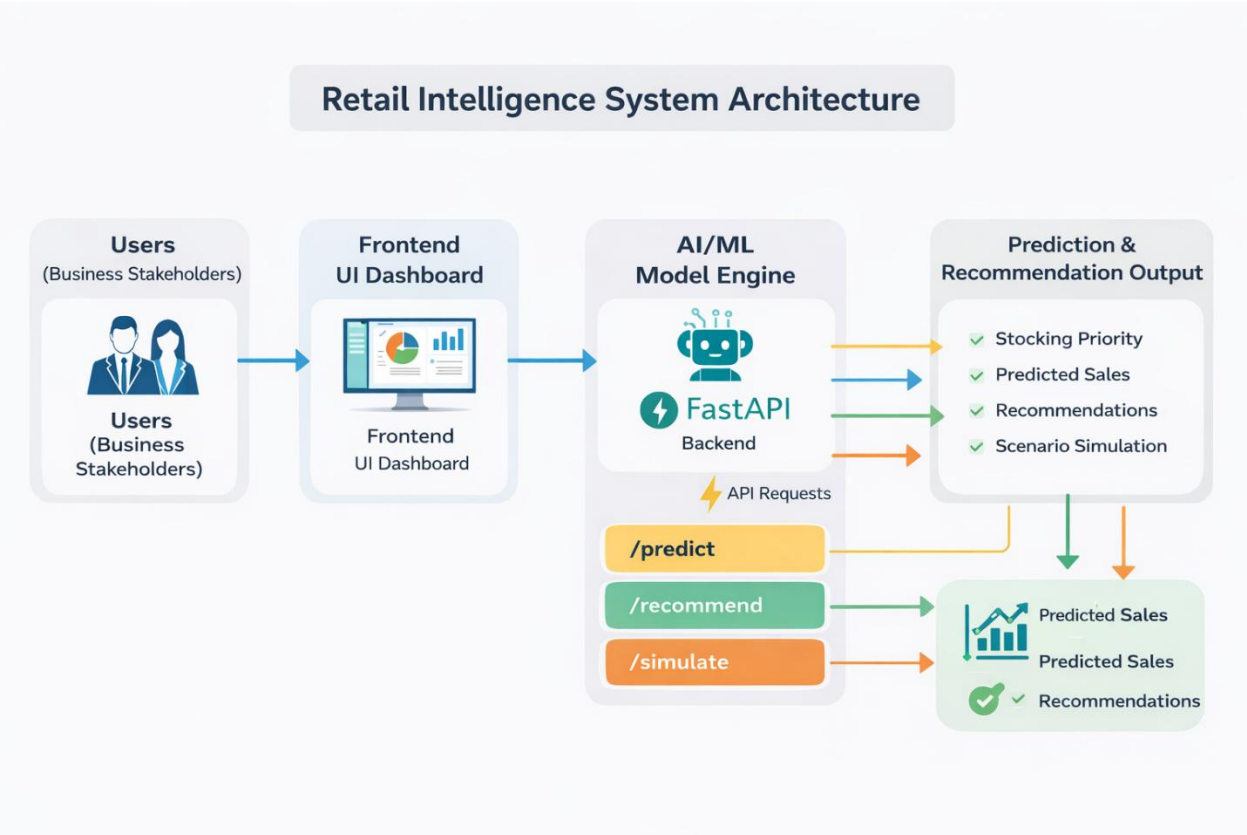
- structured features are extremely informative for the engineered business target
- adding text is still useful when representation design is handled properly
- the hybrid model offers the best practical balance for live deployment and academic explanation

13. System Architecture

The final capstone is not only a modeling exercise; it is an integrated software system. The architecture combines data processing, model inference, business-logic translation, API exposure, and a dashboard-oriented frontend.

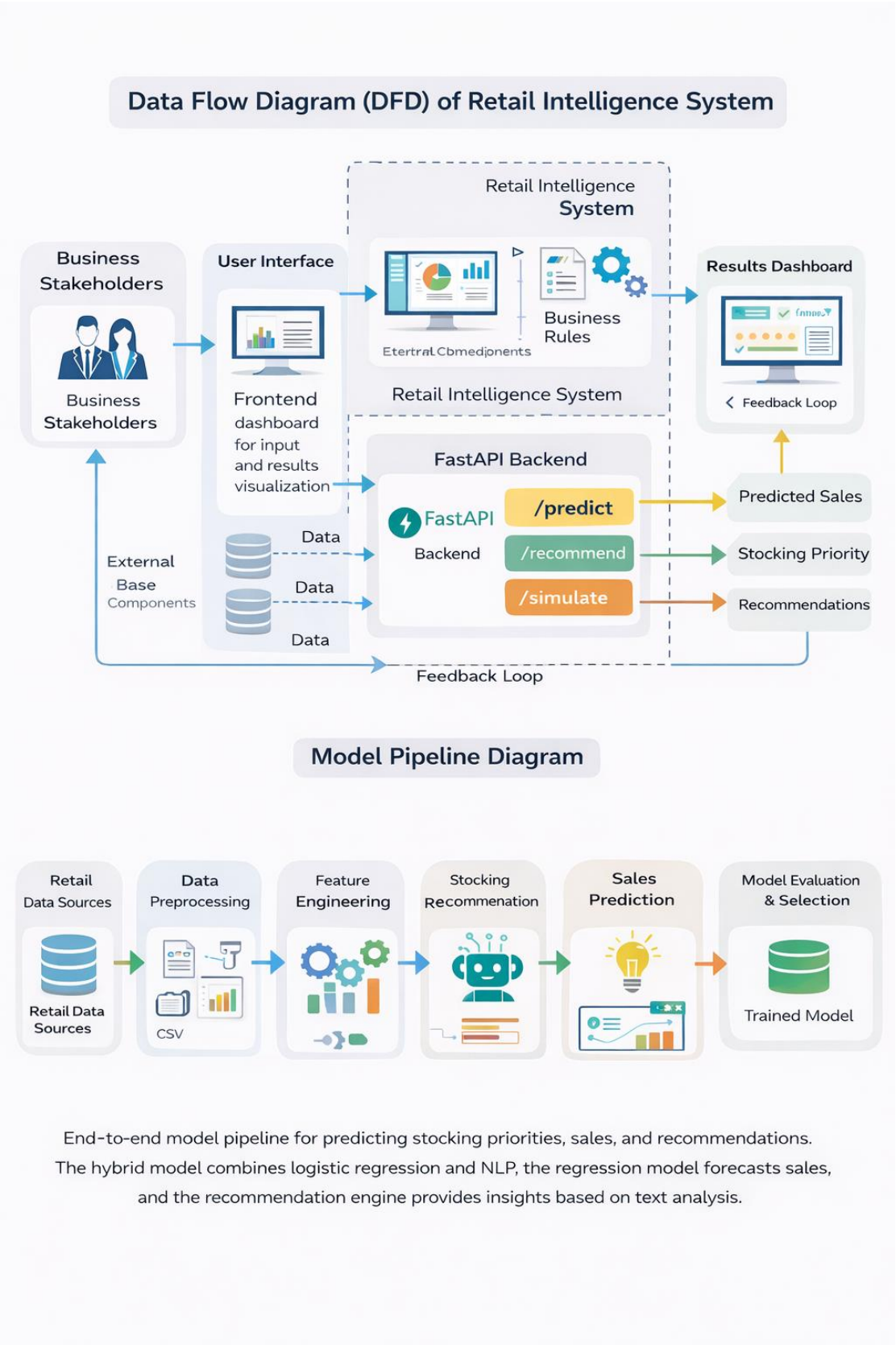
Overall architecture is depicted in the diagram below.

Figure 15: Retail Intelligence System Architecture



Also provided below depicted are the Data Flow and the Model Pipelines.

Figure 16: Data Flow and Model Pipeline



13.1 Layered architecture

Table 8: Architecture Layers

Layer	Main Components	Purpose
Data and artifacts	data/raw/, data/processed/, models/	Store raw data, precomputed insights, and serialized runtime artifacts
Model layer	priority classifier, sales regressor, scenario simulator, recommender	Produce predictions and translate them into business metrics
API layer	FastAPI routes in app/main.py	Expose prediction, recommendation, history, and insights endpoints
Presentation layer	Jinja template, JavaScript dashboard, CSS styling	Provide a business-friendly UI for demonstrations

The layered organization makes the project easier to explain and maintain. It also aligns well with academic expectations because it clearly separates modeling logic from software delivery logic.

13.2 API design

The FastAPI application exposes the following key routes:

Table 9: FastAPI Routes

Endpoint	Method	Purpose
/predict	POST	Predict Low/Medium/High stocking priority
/predict/sales	POST	Predict daily units sold and business metrics
/recommend	POST	Rank items under business-specific objectives
/query/historical	POST	Filter historical data for evidence and comparisons
/simulate	POST	Compare one product across multiple business scenarios

/insights/summary	GET	Dataset and modeling summary
/insights/model-performance	GET	Precomputed evaluation tables
/insights/feature-importance	GET	Coefficient-based and semantic drivers
/insights/business-findings	GET	Business findings from saved artifacts
/insights/model-comparison	GET	Week 4 model-lab benchmark summary
/predict/compare-models	POST	Live scenario comparison across available classification models
/ui and /demo	GET	Frontend interface and demo landing page

This route design is important because it maps directly onto the business workflow demonstrated in the frontend: input, prediction, recommendation, and insight.

13.3 Frontend architecture

The frontend is implemented inside the same FastAPI project using:

- a Jinja template for layout
- JavaScript for orchestration, API calls, and rendering
- CSS for dashboard styling

The upgraded interface includes a **Business Demo Dashboard** that sits on top of the technical views. This is a strong architectural choice for a capstone because it preserves the technical transparency of the API while also making the system intelligible to a non-technical coordinator in under a minute.

13.4 Runtime resilience

One practical engineering improvement in the final system is its resilience to serialized model incompatibility. If a saved scikit-learn artifact cannot be loaded because of an environment mismatch, the application can rebuild the operational model from the local dataset rather than failing outright.

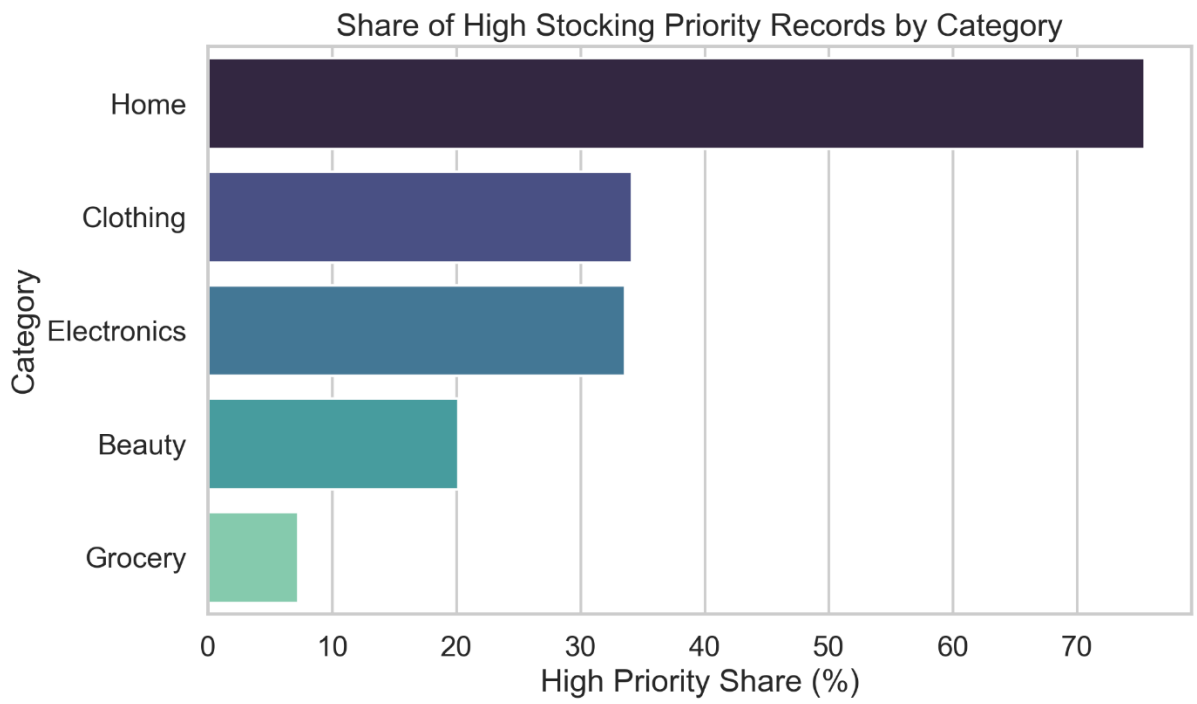
This matters in an academic submission context because it reduces the risk of a live demonstration failing for non-conceptual reasons.

14. Results and Business Insights

The final value of the project is not only that it produces predictive scores, but that it translates those scores into business insight.

14.1 Category-level stocking-priority patterns

Figure 17: Share of High Stocking Priority Records by Category



The strongest category-level result is the dominance of **Home**, where approximately **75.45%** of records fall into the High-priority class. In contrast, **Grocery** has only **7.24%** High-priority records. This difference cannot be explained by demand alone, because grocery also has the highest average units sold. The deeper explanation lies in operational context: Home products have very long average shelf life (**2,222 days**) and low spoilage risk, while Grocery has an average shelf life of only **42.65 days**.

This is an important business insight. It shows that the system is doing more than predicting movement. It is identifying products whose commercial upside can actually be converted into reliable inventory decisions.

14.2 Profit density and commercial efficiency

The artifact summaries show that **Beauty** has the highest average profit density at **31.51**, followed by **Clothing (28.07)** and **Electronics (25.79)**. Yet Beauty does not dominate the High-priority class in the same way Home does. This means that profit density alone is insufficient. A category may be commercially efficient but still be limited by other operational factors such as shelf life or replenishment variability.

This finding justifies the multi-factor recommendation engine. It would be overly simplistic to rank products only by gross profit or only by demand.

14.3 Operational risk signals

The risk findings from the final artifacts are especially revealing:

- **8.1%** of records fall below the short shelf-life threshold used by the business-insight service
- **33.45%** of records fall below the low fill-rate threshold
- **38.75%** of records trigger at least one major risk condition

These values explain why the frontend surfaces caution flags so prominently. In a dataset where more than one-third of records carry operational risk, a demand-only dashboard would be academically incomplete and practically misleading.

14.4 Interpretable business drivers

The coefficient-based analysis of the deployed linear models produces clear business levers.

Drivers of high stocking priority

Table 10: Drivers of high stocking priority

Feature	Coefficient	Business Interpretation
Fill Rate Pct	3.4453	Reliable replenishment strongly supports stocking confidence
Shelf-Life Days	3.2460	Longer shelf-life lowers spoilage risk
Current Price	1.1707	Higher-value items can support stronger stocking economics
Website Visits	0.0811	Customer attention helps reinforce priority

Drivers of low stocking priority

Table 11: Drivers of low stocking priority

Feature	Coefficient	Business Interpretation
Supplier Delay Days	1.3503	Delays weaken confidence in maintaining availability
Competitor Price	1.0146	Competitive pricing pressure affects attractiveness
Discount Percentage	0.7038	Discounting alone does not guarantee a better stocking decision
Shelf Capacity	0.2812	Large space requirements make an item less efficient

Drivers of higher sales

Table 12: Drivers of higher sales

Feature	Coefficient	Business Interpretation
Footfall Index	0.1032	Customer traffic directly supports volume
Competitor Price	0.0405	Relative pricing matters for conversion
Loyalty Program Usage Count	0.0394	Repeat-customer engagement contributes to sales
Avg Basket Size	0.0242	Larger baskets reflect richer shopping missions

14.5 Business interpretation of the integrated system

When the entire system is viewed end-to-end, three major business lessons emerge.

First, **demand and stocking priority are related but not identical**. Grocery can move quickly, but perishability and low fill-rate risk can still make it a weaker stocking candidate than more stable categories.

Second, **semantic product information is useful when it is structured carefully**. Text-only modeling was weak, but the hybrid model showed that semantic retail context can complement structured variables when encoded through a column-aware template.

Third, **deployment-quality systems require more than good notebook metrics**. The capstone matured when it moved from exploratory perfection caused by leakage to realistic, leakage-aware models that can run live inside FastAPI and explain themselves through a dashboard.

15. Conclusion

This capstone project successfully transforms a basic retail forecasting exercise into a much richer retail intelligence system for inventory optimization. The work begins with rigorous exploratory analysis, progresses through extensive feature engineering and model experimentation, and culminates in a deployable FastAPI application with a business-oriented frontend.

Several achievements make the final system academically defensible and practically convincing.

1. The project clearly identifies that retail stocking decisions cannot be reduced to sales alone.
2. It constructs a business-oriented target that combines profitability, shelf efficiency, and operational stability.
3. It evaluates multiple classical and modern model families rather than relying on a single convenient baseline.
4. It explicitly recognizes and corrects feature leakage in early regression experiments.
5. It deploys stable, interpretable models that can support live demonstration and business explanation.

The final deployed architecture is well aligned with the capstone objective. The hybrid logistic regression model supports business-aware stocking-priority classification, while the Ridge-based sales model provides realistic demand estimation and downstream profit context. Together, these models power a recommendation and insight layer that is more useful than isolated prediction endpoints.

From an academic standpoint, the strongest aspect of the project is not a single metric value. It is the quality of the overall reasoning: the project shows how to move from EDA to feature engineering, from baseline models to deployment-aware model selection, and from raw predictions to actionable business interpretation.

16. Future Work

The project establishes a strong foundation, but several extensions would further improve both academic rigor and production readiness.

16.1 Methodological improvements

1. unify the operational evaluation pipeline and the Week 4 model-lab benchmark under a single formal protocol so cross-table comparisons become fully consistent
2. perform more systematic hyperparameter tuning for both the classifier and regressor
3. use time-aware validation for the sales model instead of only random train-test splitting
4. add ablation studies showing the marginal contribution of text features, traffic features, and operational features

16.2 Modeling extensions

1. revisit deep learning only after establishing a stronger leakage-free structured baseline for hybrid tabular-text learning
2. explore gradient boosting or CatBoost-style regression for demand prediction
3. incorporate calibrated probabilities for the stocking-priority classifier
4. extend the scenario simulator with promotional elasticity and sensitivity analysis

16.3 System and product extensions

1. add downloadable executive reports from the recommendation engine
2. add frontend smoke tests and broader automated end-to-end testing
3. enable live artifact validation and model warm-up before demonstrations
4. integrate richer user roles so the same backend can serve both technical reviewers and business stakeholders

In summary, the current capstone already demonstrates a complete and well-reasoned retail intelligence system. Future work should build on this by tightening evaluation discipline, extending experimentation, and expanding the reporting capabilities of the deployed application.

17. Bibliography / References

1. Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., et al. (2011).
   Scikit-learn: Machine Learning in Python.
   Journal of Machine Learning Research, 12, 2825–2830.
2. Friedman, J. H. (2001).
   Greedy Function Approximation: A Gradient Boosting Machine.
   Annals of Statistics, 29(5), 1189–1232.
3. Ho, T. K. (1995).
   Random Decision Forests.
   Proceedings of the Third International Conference on Document Analysis and Recognition.
4. Tibshirani, R. (1996).
   Regression Shrinkage and Selection via the Lasso.
   Journal of the Royal Statistical Society, Series B.
5. Hoerl, A. E., & Kennard, R. W. (1970).
   Ridge Regression: Biased Estimation for Nonorthogonal Problems.
   Technometrics, 12(1), 55–67.
6. Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., et al. (2017).
   Attention Is All You Need.
   Advances in Neural Information Processing Systems (NeurIPS).
7. Devlin, J., Chang, M. W., Lee, K., & Toutanova, K. (2019).
   BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding.
   NAACL-HLT.
8. Chollet, F. (2015).
   Keras.
   https://keras.io
9. Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., et al. (2019).
   PyTorch: An Imperative Style, High-Performance Deep Learning Library.
   NeurIPS.
10. McKinney, W. (2010).
    Data Structures for Statistical Computing in Python.
    Proceedings of the 9th Python in Science Conference.
11. Hunter, J. D. (2007).
    Matplotlib: A 2D Graphics Environment.
    Computing in Science & Engineering.
12. FastAPI Documentation. (2024).
    FastAPI Framework for Building APIs.
    https://fastapi.tiangolo.com
13. Uvicorn Documentation. (2024).
    ASGI Server Implementation.
    https://www.uvicorn.org
14. Pedregosa et al. (2023).
    Scikit-learn Documentation.

https://scikit-learn.org

15. Hugging Face. (2024).
Transformers Library Documentation.
https://huggingface.co/docs/transformers

16. Géron, A. (2019).
Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow.
O'Reilly Media.

17. Aggarwal, C. C. (2018).
Machine Learning for Text.
Springer.

18. Duly Completed Checklist

Is the Cover page in proper format?	Y / N
Is the Title page in proper format?	Y / N
Is the Certificate from the Mentor in proper format? Has it been signed?	Y / N
Is Abstract included in the Report? Is it properly written?	Y / N
Does the Table of Contents page include chapter page numbers?	Y / N
Does the Report contain a summary of the literature survey?	Y / N
Are the Pages numbered properly?	Y / N
Are the Figures numbered properly?	Y / N
Are the Tables numbered properly?	Y / N
Are the Captions for the Figures and Tables proper?	Y / N
Are the Appendices numbered?	Y / N
Does the Report have Conclusion / Recommendations of the work?	Y / N
Are References/Bibliography given in the Report?	Y / N
Have the References been cited in the Report?	Y / N
Is the citation of References / Bibliography in proper format?	Y / N

Note: A copy of this checklist should be included in the last page of the Final report. This checklist, duly completed and signed by the student, should also be verified and signed by the evaluators. Evaluators are requested to ensure that the students have submitted their reports properly.